Performance Investigation

What was actually changed for speed, what was deliberately left alone, and what was tuned with no change needed. Every number below was measured, not estimated.

01 The tan/atan elimination

`forward_search/kernels/forward_search.cl` (both kernel variants) · `forward_search/src/forward_search.cpp`

The problem

For each of the 35,721 candidate poses, the search kernel samples 1000 symmetric angle pairs. The original code called `tan()` twice per sample to get `tan(theta_0 + dtheta)` and `tan(theta_0 - dtheta)`:

```
2 × 1000 samples × 35,721 candidates ≈ 71,000,000 tan() calls, plus one atan() call per candidate
```

The fix: two things, done together

1. **Precompute `tan(dtheta)` once.** The sampled angles don't depend on which pose is being tested, so this is identical for every candidate: 1000 calls total instead of 71 million. The kernel then derives `tan(theta_0 ± dtheta)` via the tangent addition formula, pure multiply/add/divide, no trig call at all.
2. **Skip `atan()` entirely.** `theta_0` turned out to never be used as an angle anywhere else in the kernel, only ever as `tan(theta_0)`. Since `tan(atan(x)) == x`, that's just `x_p / z_p` directly, a bonus beyond what was originally planned.

```
tan(theta_0 + dtheta) = (tan_theta0 + tan_dtheta) / (1 - tan_theta0 · tan_dtheta)
tan(theta_0 - dtheta) = (tan_theta0 - tan_dtheta) / (1 + tan_theta0 · tan_dtheta)
```

BEFORE		AFTER
52.2ms	→ 1.6× faster	**32.9ms**

ⓘ **Verified against:** `test_kernel_timing.py`, written first to require real device-side kernel timing before that existed. Whole-program wall time is dominated by HDF5 I/O and CPU-side sinogram building, so this test is what made an honest before/after possible. Existing tests stayed green throughout, since this is a refactor, not a behavior change.

02 Build flags, work-group sizing, and buffer caching

Three more fixes, same before/after treatment.

Build flags

The problem. `buildSinogram()` is a tight CPU-side pixel loop that was compiling with no optimization flags at all: no inlining, no vectorization, no loop unrolling. That gap against an optimized build routinely lands in the 100 to 300× range.

The fix. Added in two separate places, since the CLI binary and the pybind11 interface use two independent build paths: `buildtype=release` added to `meson.build` for the CLI, and `-O3` added to `backend.py`'s JIT compile flags for the pybind path (which JIT-compiles on import with its own flag list, entirely separate from meson).

✓ **Verified against:** stage timing added directly around `buildSinogram()`, compared with the search kernel's own ~48ms, on the real dataset. Measured ~11.4s with no optimization flags, ~0.3 to 0.8s with `-O3`: roughly 240×.

Work-group sizing

The problem. `CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE` is a granularity hint (pick a work-group size that's a multiple of this, to match the hardware's SIMD width), not a recommended size. Using it directly as the work-group size, floored at 64, produced work-groups far smaller than the device can actually hold.

The fix. Switched to `CL_DEVICE_MAX_WORK_GROUP_SIZE` (256 on this GPU), matching `projection-center`'s own kernel sizing.

ⓘ **Verified against:** nothing isolated. Found not to be the dominant factor in the speed gap it was chasing, once the build flags fix above was applied, and at ~48ms per search the difference is no longer visible. Kept for parity with `projection-center`'s own sizing, not because of a measured speedup of its own.

`resample()` buffer/kernel caching

The problem. `resample()` streams the dataset in batches, and each batch needs GPU buffers plus a kernel bound to them via `setArg()`. Without caching, every batch call allocated fresh `cl.Buffer` objects from scratch, a genuinely expensive driver-level operation, and re-bound the kernel args, on every single batch.

The fix. Hoists that invariant setup out of the loop: buffers are allocated once and reused across every batch, with only their contents overwritten via `clEnqueueWriteBuffer`.

✓ **Verified against:** measured on the real dataset, 48s down to 3.9s (12×), with resampled output confirmed unchanged.

03 Batch-size tuning for resample

`--batch-size` controls how many projections `resample` uploads to the GPU per dispatch (default 16). Swept the whole range on the real dataset, same data, same pose, back to back.

--BATCH-SIZE	TIME		BATCHES
4	11.1s		45
8	6.5s		23
16 (default)	**6.6s**		**12**
32	11.5s		6
64	11.5s		3
128	10.1s		2
180 (one batch)*	43.9s		1

*Measured separately, possibly under different disk-cache conditions than the other six values (which were measured back to back). Still clearly the slowest of the set.

Not "bigger is better up to a point"

There's a real sweet spot around 8 to 16, and performance gets worse moving *either* direction away from it. Below that, per-dispatch overhead (GPU buffer setup, host/device transfer setup, kernel launch, all paid once per batch) stops being amortized well. Above it, a single large transfer stops overlapping with compute as well as several smaller ones do, and 180 in one shot is dramatically worse.

Conclusion: the existing default of 16 is already in the good zone. 8 is close enough to call it noise; every other value tested was clearly worse, some by close to 2×. No change was made because none was needed.